



Tech Day

Trust, but Verify: Verifying Red Hat Container Images to Assure Platform Trust

Gareth Healy / Dan Hawker





Gareth Healy

Principal Architect
Red Hat UK&I Services



Dan Hawker

Principal Architect
Red Hat UK&I Services



What we will show and tell today

Tell..

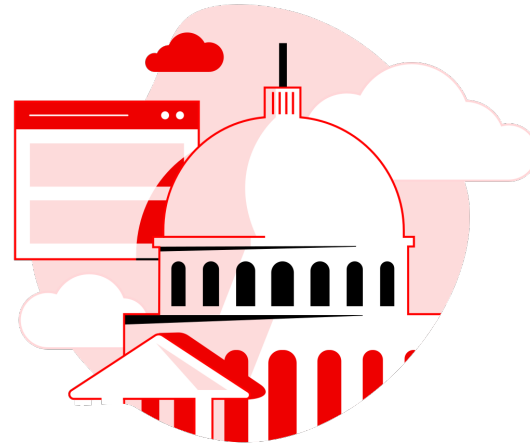
- ▶ What do we mean by "Trust but Verify"?
- ▶ Why is this important and How have people been doing this?
- ▶ Introduction to Trusted Software Supply Chain
- ▶ How to sign and verify your own images
- ▶ Lessons learned from implementing in the wild

Show...

- ▶ How to start verifying Red Hat Content via
 - OpenShift
 - Podman, Buildah, Skopeo and oc-mirror



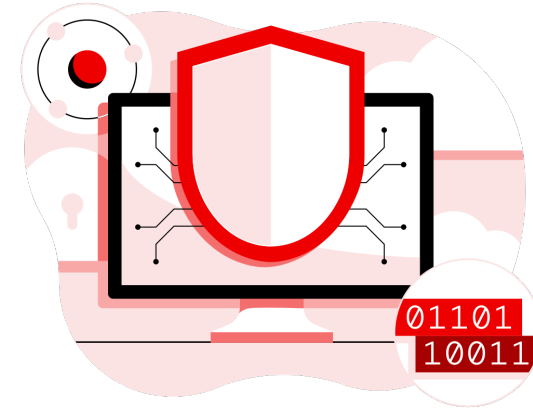
What do we mean by “Trust but Verify”?



- ▶ Trust is inferred between you and Red Hat, if you are a customer
- ▶ Trust doesn't stop bad-actors either within Red Hat, between Red Hat and your network or within you own network; *verification* does
- ▶ You've been automatically verifying RHEL RPMs for years, *even if you didn't realise*, so why should containers be any different?



Why is this important?

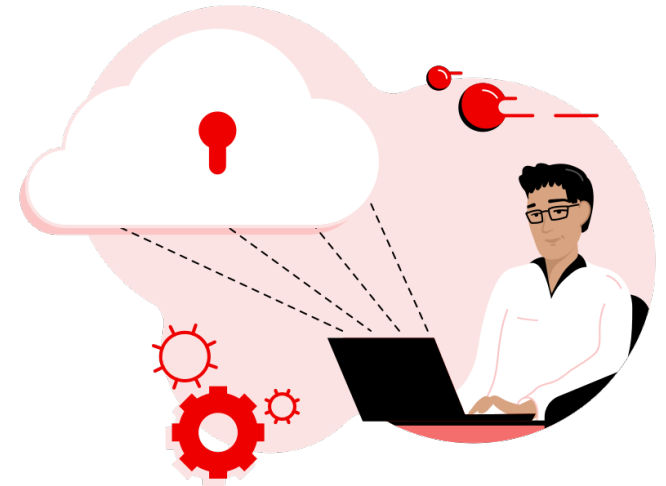


- ▶ Weekly occurrences of supply chain attacks and typosquatting which means developers are constantly under attack and run the risk of accidentally using malicious packages
- ▶ Access tokens are regularly stolen/leaked and typically, provide the attacker access to push compromised changes
- ▶ *Open Question:* Can you *automatically* assert that the components running in your environment were built by the trusted source?



But how have people been doing this?

- ▶ Trust but verify is not a new concept in the container space:
 - Container PGP signing was supported in [OCP3](#)
 - [Notary](#) v1 was released in 2015 and has gone through multiple iterations since
 - Pull via SHA and verify remote SHA matches local SHA





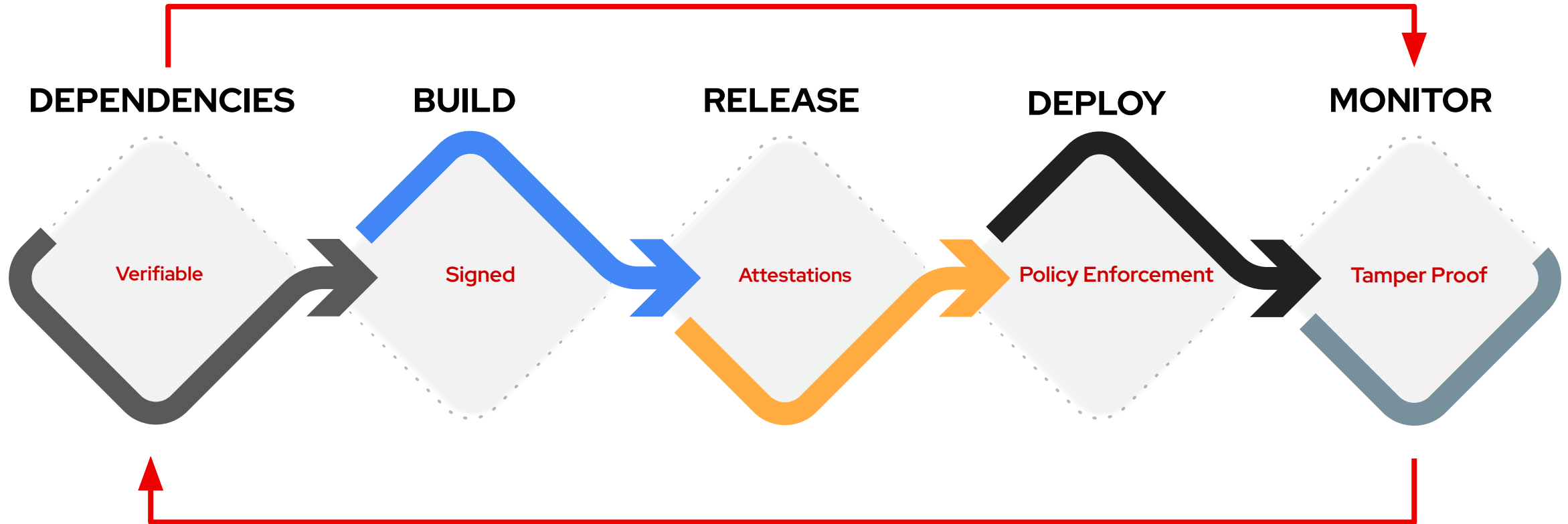
But it's 2026!

Shouldn't we be
adopting Trusted
Software Supply
Chain?





Trusted Software Supply Chain



Trusted Profile
Analyzer



Trusted Artifact
Signer



Advanced
Developer Suite



Advanced Cluster
Security



Open Source Collaboration

Innovations coming from Open Source communities:



[In-toto](#)



[SLSA](#)



[TUF](#)

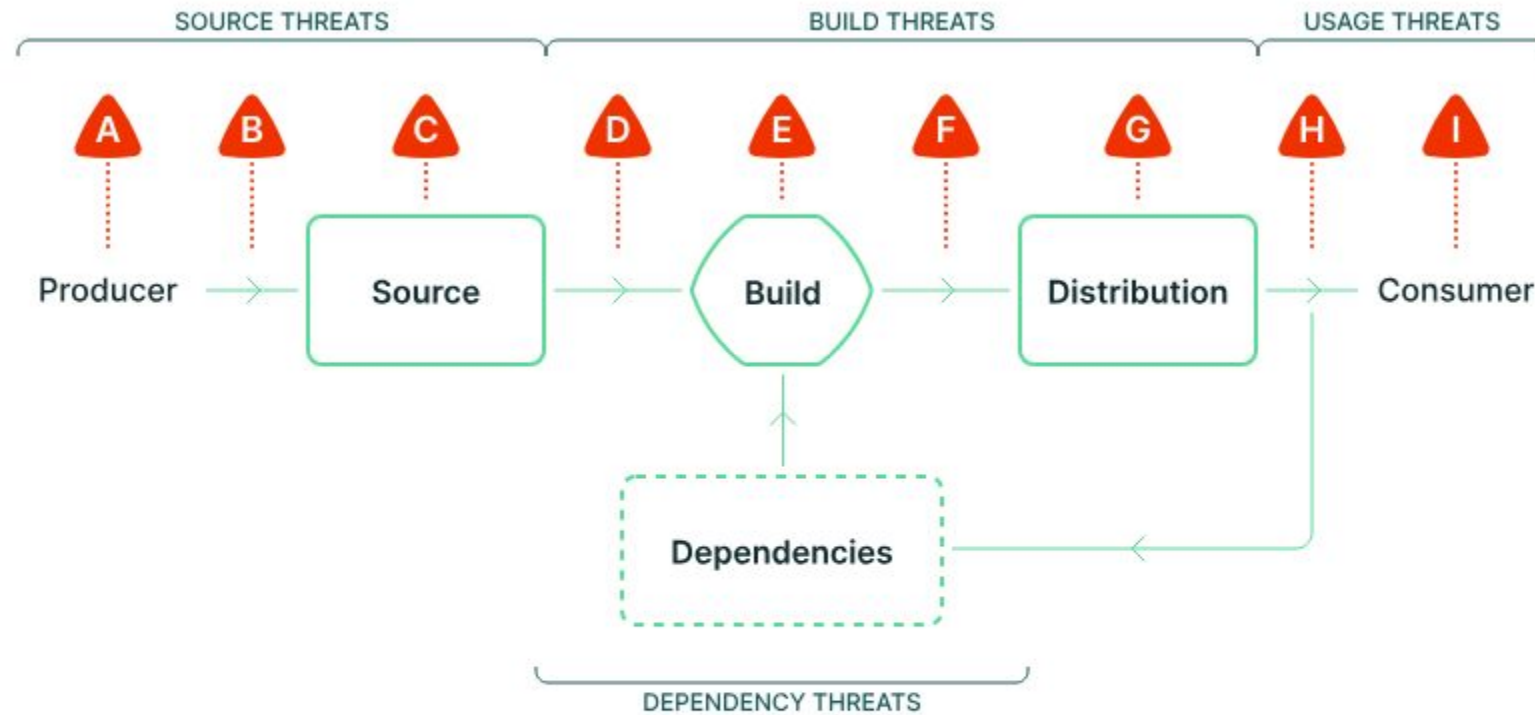


[Sigstore](#)



[OpenSSF](#)

SLSA



A Producer (entity)
B Authoring & reviewing
C Source code management

D External build parameters
E Build process
F Artifact publication

G Distribution channel
H Package selection
I Usage



SLSA Levels

- ▶ SLSA 1
 - Provenance showing how the package was built
- ▶ SLSA 2
 - Signed provenance, generated by a hosted build platform
- ▶ SLSA 3
 - Hardened build platform

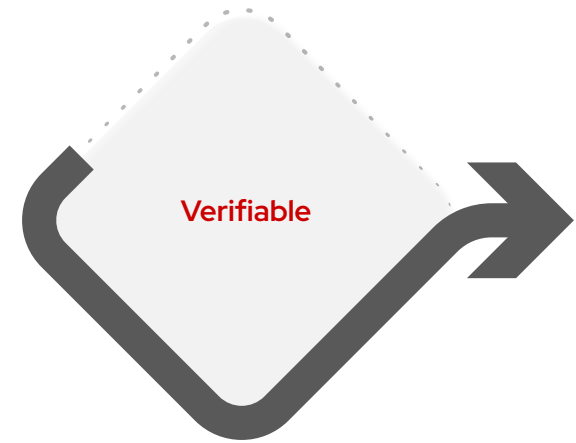


SLSA Guiding Principles

- ▶ Simple levels with clear outcomes
- ▶ Trust platforms, verify artifacts
- ▶ Trust code, not individuals
- ▶ Prefer attestations over inferences



Verification is the first step



- ▶ Red Hat *aims* to generate SLSA Level 3 Containers
- ▶ Trusted Software Supply Chain covers many areas, the easiest place to start is by verifying vendors content

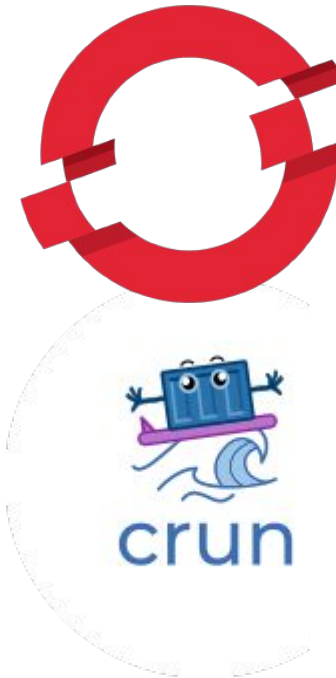




OK, how do i verify
Red Hat Containers?



Verifying via OpenShift



[ClusterImagePolicy](#) and ImagePolicy introduced in OCP [4.18](#) as TP and GA'ed in OCP [4.20](#)

```
gahealy@gahealy-mac: ~/Documents/Git/github.com/garethahealy/verifying-redhat-images
apiVersion: config.openshift.io/v1alpha1
kind: ClusterImagePolicy
metadata:
  name: openshift
spec:
  scopes:
    - quay.io/openshift-release-dev/ocp-release
  policy:
    rootOfTrust:
      policyType: PublicKey
      publicKey:
        keyData: LS0tLS1CRUdJTiBQVUJMSUMgS0VZLS0tLS0KTUJlJQ01qQU5CZ2txaGtpRz13MEJBUEUvGQUFPQ0FnoEFNSU1DQ2dLQ0FnRUE
wQVNSdUgyVEExXkdJVcVBIWjRjcAo3NWc3RW5jQmtnUUhkSm5qenBVzVLUVRNAC9zaUJvQi9Cb1Nydg1QTXduQ2hiVENuUU9JUWVadURpRmSodUo
3Ck0vRDNiN0pvWDBtMTIzTmNDU242N21BZGpCYTZCZzZrdWtaZ0NQNFpVWmVWU2FqV1gvRWp5bEZjUkZPWFc1N3AKUkRDRU40Mkova1lsVnF0K2c
5K0dya2VyOFN6ODZIM2wwdGJxT2RqYnovVnhIWWh3RjBjdFVNSHN5V1JEcTJRUApcXp0WGxtbE1oUy9Qb0ZyNlI0dS83SEnuL0srTGVnY08yZkF
GT2I0MEt2S1NLS1ZENmx1d1VaRXJob3AxQ2dKClhqRHRHbW1POWRHTUY3MW1mNkhFZmFLU2R5K0VFNm1TRjJBMLZ20VFOqMf3TWlxMmtPekVpTGc
0bkFkS1Q4d2cKWhJNQW1QQ3FHSXNYTkdaNC9RK1lUd3dsY2UzZ2xxYjVMOXRmTm96RWRTUj10ODVERVNmUuXRRWRZM0NhbHdLTQpCVDFPRWhFWDF
3SFJDVTRkck1PZW02Qk5XMFZ0c2NHdEhtQ3JzNzRqUGV6aHd0VDh5cGt5UytUMHpUNFRzeTZmClZYa0o4WVNIeWVuU3pNQjJPcDJidnNFM2dyWSt
zNzRXAcESVU1BNkRCeGNuUaUxNU5Tek3Znphb05XT0RjTEYKcDdCWThhYUhfMk1xRnhZRlgrSWJqcGtRUmZhZVFRc291REZkQ2tYRUZWZ1BwYkQ
yZGs2Rmx1YU1UHV5eHRJVApna1ZFdEdRSzJxR0NGR21RSEZkNGhmVitlQ0EZM0pybzF6MHPvQk01QmJJJSVEZK2VWRnd0M0FsWnA1VZ3cjZkCnN
lY3FraS9Scm12M1kwZHF0VZPbjNVQ0F3RUFBU09C10tLS0tRU5EIFBVQkxJQyBLRVktLS0tLQo=
→ verifying-redhat-images git:(main) ✖
```



Verifying via Podman, Buildah, Skopeo and oc-mirror

The OCP feature piggy-backs the [containers/image](#) feature used by cri-o, which means Podman, Buildah, Skopeo and oc-mirror can all use the same config as well!

```
gahealy@gahealy-mac:~/Documents/Git/github.com/garethahealy/verifying-redhat-images
→ verifying-redhat-images git:(main) cat samples/HOME/.config/containers/registries.d/sigstore-registries.yaml
docker:
  registry.access.redhat.com/ubi9/openjdk-21-runtime:
    use-sigstore-attachments: true
→ verifying-redhat-images git:(main) cat samples/HOME/.config/containers/policy-ubi9.json
{
  "default": [
    {
      "type": "reject"
    }
  ],
  "transports": {
    "docker": {
      "registry.access.redhat.com/ubi9/openjdk-21-runtime": [
        {
          "type": "sigstoreSigned",
          "keyPath": "/home/runner/redhat-sigstore.pub"
        }
      ]
    }
  }
}
```

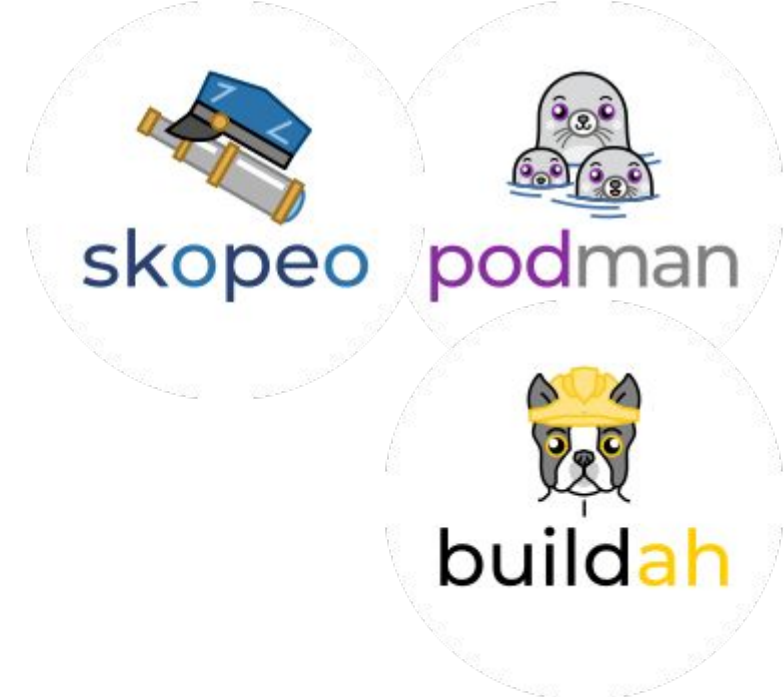


Sample Commands

```
$ podman pull registry.access.redhat.com/ubi9/openjdk-21-runtime:1.21
```

```
$ skopeo copy \  
  docker://registry.access.redhat.com/ubi9/openjdk-21-runtime:1.21 \  
  dir:/tmp/openjdk-21-runtime
```

```
$ oc-mirror --v2 --remove-signatures=false --secure-policy=true \  
  --config samples/oc-mirror/imageset.yaml --log-level debug \  
  file://$(pwd)/oc-mirror-data
```



But isn't this the same as...

```
gahealy@gahealy-mac:~/Documents/Git/github.com/garethahealy/verifying-redhat-images
→ verifying-redhat-images git:(main) x cat policy.json
{
  "default": [
    {
      "type": "insecureAcceptAnything"
    }
  ],
  "transports": {
    "docker": {
      "registry.access.redhat.com": [
        {
          "type": "signedBy",
          "keyType": "GPGKeys",
          "keyPaths": ["/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release", "/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-beta"]
        }
      ],
      "registry.redhat.io": [
        {
          "type": "signedBy",
          "keyType": "GPGKeys",
          "keyPaths": ["/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release", "/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-beta"]
        }
      ]
    },
    "docker-daemon": {
      "": [
        {
          "type": "insecureAcceptAnything"
        }
      ]
    }
  ]
}
```



Demos on GitHub

Examples and commands @ github.com/garethahealy/verifying-redhat-images

- ▶ <https://github.com/garethahealy/verifying-redhat-images/tree/main/screenrecordings>
- ▶ <https://github.com/garethahealy/verifying-redhat-images/actions/workflows/verify-redhat.yaml>

A terminal window with a dark background. The title bar shows 'gahealy@gahealy-mac: ~/Documents/Git/github.com/garethahealy/verifying-redhat-images'. The prompt is '→ verifying-redhat-images git:(main) *'. The command 'cat samples/ocp/FeatureGates.yaml' has been executed, and the output is displayed in a light blue monospace font.

```
→ verifying-redhat-images git:(main) * cat samples/ocp/FeatureGates.yaml
apiVersion: config.openshift.io/v1
kind: FeatureGate
metadata:
  name: cluster
spec:
  featureSet: TechPreviewNoUpgrade
→ verifying-redhat-images git:(main) *
```

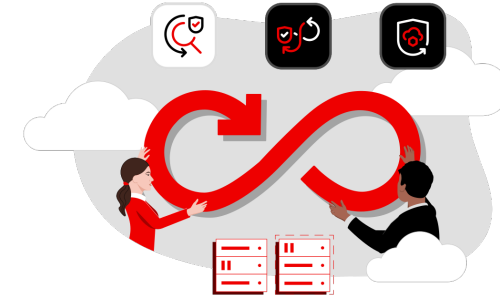




But how do i sign my
containers?



Thinking about Signing your own Images?



Products that can help:

- ▶ [Red Hat Trusted Artifact Signer](#) (RHTAS)
 - Runs on OpenShift/RHEL and based on Sigstore
 - Customer controls the PKI or OIDC for keyless
- ▶ [Red Hat Advanced Cluster Security](#) (RHACS)
 - You've signed the images with TAS, now lets verify and enforce with ACS

Have access to demo.redhat.com?

- ▶ [Red Hat Advanced Developer Suite - Software Supply Chain Demo](#)
- ▶ [Red Hat Advanced Developer Suite - Technical Workshop](#)

Have a Red Hat account manager? Speak to them on how Red Hat can help you on this journey.



Signing is cool but is binary, which is where attestations come!

- ▶ What is an Attestation?
 - Define: attestation - “evidence or proof of something”
- ▶ Attestations can provide the verifier with signed metadata on:
 - What Git commit the build is from
 - Where it was built
 - Who triggered the release
- ▶ This creates an Audit trail for when things go bad...
- ▶ And decision points on whether to allow artifacts into your environments



Lessons Learned



- ▶ Signing per environment or at a set stage in CI/CD is an anti-pattern, i.e.: signing in dev/uat/prod as control gate. Attestations are a better control method.
- ▶ Keyless signing is great for developers but if used for signing production artifacts should be used in conjunction with other security methods to stop authentication/MFA leaks.
- ▶ Generate and attach all the attestations.
- ▶ Adopting [GitSign](#) is super easy! Tried using PGP? GitSign is commit signing with zero developer headache and is supported via RHTAS.
- ▶ Read blogs from [Adnan Khan](#) and [John Stawinski](#). Security researchers focused on supply chain attacks.



Final Thoughts

- ▶ You are going to start verifying Red Hat content... right?
- ▶ But what's the health of the other artifacts you are pulling into your ecosystem, such as Java/Python/npm/Container Bases/etc?
- ▶ If you are generating metadata (i.e.: SBOMS); are they signed and verifiable?
- ▶ If you are already signing; is the *process* tamper-proof?



Previously asked Questions

- ▶ But, verifying Red Hat payloads isn't the trusted supply chain this is just delivery part...
 - No, but it's part of a bigger story. You have to start somewhere and verifying is the easiest starting point for customers
- ▶ But, this doesn't guarantee what's inside the container...
 - No, verifying the container on its own does not, but Red Hat provides SLSA level 3 payloads
- ▶ But, what if Red Hat lost control of the signing keys, my cluster would be bricked!
 - No, but you would need to upgrade to the next version for the newly signed payloads
- ▶ Can I use ClusterImagePolicy/ImagePolicy for my own images?
 - Yes, but depending on how you sign your containers; ACS might be a better option
- ▶ How is ClusterImagePolicy/ImagePolicy different to an ACS policy to verify image signatures?
 - ImagePolicy is via node config and ACS is via an Admission Controller and/or ACS Sensor



Cont...

- ▶ But, who creates the attestations and how do we trust them?
 - Automated by your pipeline, attached to the container via an OCI reference and signed.
- ▶ But, developers could self-sign and generate this metadata?
 - You still need to follow standard authorisation processes to only allow service accounts access to your container registry and your key management vault.





Questions?

Gareth Healy: gahealy@redhat.com

Dan Hawker: dhawker@redhat.com

Registration is open!



Connect

London | 8 October 2026



Appendix

- ▶ Renovatebot
 - Remove the churn of updating dependency versions
 - Pin container base images, GitHub Actions and Gitlab Pipeline Includes
 - Can be part of an *Evergreening* approach
- ▶ Step-Security harden-runner GitHub Action
 - Detect egress and source code changes at pipeline run
 - Discovered several supply chain attacks against:
 - tj-actions/changed-files compromise (CVE-2025-30066)
 - Google's Open-Source Project Flank
 - Microsoft's Open-Source Project Azure Karpenter Provider in Real-Time
 - Popular Nx Build System Package Compromised with Data-Stealing Malware





Thank you!

Red Hat is the world's leading provider of enterprise open source software solutions. Award-winning support, training, and consulting services make Red Hat a trusted adviser to the Fortune 500.



 [linkedin.com/company/red-hat](https://www.linkedin.com/company/red-hat)

 [facebook.com/redhatinc](https://www.facebook.com/redhatinc)

 [youtube.com/user/RedHatVideos](https://www.youtube.com/user/RedHatVideos)

 twitter.com/RedHat